

Санкт-Петербургский государственный университет

Парфенов Данил Игоревич

Выпускная квалификационная работа

Разработка байткода и виртуальной машины для языка Lama

Уровень образования: бакалавриат

Направление *09.03.04 «Программная инженерия»*

Основная образовательная программа *СВ.5080.2022 «Программная инженерия»*

Научный руководитель:
доцент кафедры системного программирования, к. ф.-м. н., Булычев Д. Ю.

Рецензент:
Инженер-программист 1 категории, АО "НПО Эшелон", к. ф.-м. н., Лозов П.А.

Санкт-Петербург
2026

Saint Petersburg State University

Danil Parfenov

Bachelor's Thesis

Development of bytecode and a virtual machine for the Lama programming language

Education level: bachelor

Speciality *09.03.04 "Software Engineering"*

Programme *CB.5080.2022 "Software Engineering"*

Scientific supervisor:

C.Sc, docent. D.Y. Boulytchev

Reviewer:

Category II Software Engineer at "JSC «NPO Echelon»", C.Sc P.A. Lozov

Saint Petersburg
2026

Оглавление

Введение	4
1. Постановка задачи	6
2. Обзор	7
2.1. Обзор языка Lama	7
2.2. Существующая инфраструктура языка Lama	8
2.3. Ограничения существующего интерпретатора	9
2.4. Подходы к организации виртуальных машин	10
2.5. Выводы	12
3. Архитектура	13
4. Описание реализации	16
4.1. Прототип виртуальной машины	16
4.2. Доработка генерации байткода	17
4.3. Загрузка и связывание модулей	18
4.4. Конвертация байткода во внутреннее представление . . .	20
5. Тестирование и оценка производительности	25
5.1. Методика тестирования	25
5.2. Оценка производительности	26
5.3. Анализ выявленных ошибок и ограничений	27
Заключение	28
Список литературы	29

Введение

Современные языки программирования представляют собой не только синтаксис и набор семантических правил, но и программную инфраструктуру, обеспечивающую выполнение написанных на них программ [3, 4]. В такую инфраструктуру обычно входят компилятор, промежуточные представления программы, среда исполнения и один или несколько механизмов запуска программы. От полноты и согласованности этих компонентов зависит не только удобство использования языка, но и возможность его дальнейшего развития.

Одним из распространенных подходов к организации исполнения программ является использование байткода и виртуальной машины [6, 10, 5]. В такой архитектуре исходная программа сначала преобразуется в промежуточное низкоуровневое представление, не привязанное напрямую к конкретному процессору, а затем выполняется специальной средой исполнения. Этот подход используется как в промышленных языках программирования, так и в учебных и исследовательских проектах, поскольку он позволяет отделить компиляцию от конкретного способа выполнения программы.

Lama является учебным языком программирования, разрабатываемый для знакомства с областью языков программирования, компиляторов и инструментов [8, 2]. Язык сочетает императивные и функциональные конструкции: переменные, функции, рекурсию, замыкания, массивы, строки, S-выражения и сопоставление с образцом. В существующей реализации Lama исходная программа компилируется в промежуточное стековое представление, после чего может быть либо скомпилирована в нативный код, либо в байткод.

Тем не менее, на данный момент не существует стандартного интерпретатора для байткода. Кроме того, формат байткода неспецифицированный и его запуск многомодульных программ из байткода технически невозможно реализовать.

Перечисленные проблемы делают необходимой разработку новой виртуальной машины, реализованной ближе к существующей среде

исполнения языка. Такая виртуальная машина должна использовать представления значений Lama, взаимодействовать со средой исполнения, вызывать функции и поддерживать загрузку нескольких байткодных модулей. В рамках данной работы требуется доработать байткодное представление и разработать виртуальную машину для языка Lama, реализовать основные компоненты загрузки, связывания, валидации и исполнения, а также подготовить инфраструктуру проверки корректности полученного решения. Предлагаемый подход должен повысить полноту поддержки байткодного исполнения по сравнению с существующим интерпретатором, упростить дальнейшее развитие среды выполнения и создать основу для последующих исследований оптимизации исполнения программ.

1. Постановка задачи

Целью данной работы является разработка виртуальной машины байткода языка программирования Lama, согласованную с существующим компилятором в ассемблер x84_64. Для достижения этой цели были сформулированы следующие задачи.

1. Реализовать прототип виртуальной машины для существующего формата представления байткода Lama
2. Доработать формат представления байткода Lama и прототип виртуальной машины для возможности исполнения многомодульных программ
3. Реализовать виртуальную машину на основе прототипа с предварительным анализом байткода для ускорения выполнения
4. Проверить корректность работы виртуальной машины и оценить её производительность на существующих тестах

2. Обзор

В этом разделе рассматривается контекст, необходимый для формулировки требований к байткодной виртуальной машине Lama. Сначала кратко описываются свойства самого языка, которые влияют на исполнение программ: единое представление значений, работа с составными объектами, функции, замыкания и модульность. Затем разбирается существующая инфраструктура языка Lama. В завершение обсуждаются ограничения текущего интерпретатора и общие архитектурные решения, применимые к стековым виртуальным машинам.

2.1. Обзор языка Lama

Lama — учебный язык программирования, разработанный для изучения устройства языков программирования, компиляторов и сред исполнения. Описание языка и его открытая реализация доступны в репозитории проекта и спецификации языка [8, 2].

В языке присутствуют различного рода конструкции: изменяемые переменные, операторы присваивания, условные операторы, циклы и последовательное исполнение выражений, функции как значения. Кроме того, в языке есть составные структуры данных: массивы, строки, S-выражения и сопоставление с образцом.

Исходный байткод Lama задаёт стековую модель вычислений: инструкции берут аргументы со стека и помещают результат обратно на стек. На уровне выполнения программа Lama оперирует единым представлением значений. Часть значений может храниться непосредственно на стеке, а составные объекты размещаются в куче и обслуживаются средой выполнения.

Отдельно важна модульная система. Практическая программа на Lama может состоять из нескольких модулей, импортирующих функции и значения друг друга. Это влияет на байткодное исполнение: запуск программы в общем случае связан не только с одним файлом, но и с загрузкой нескольких модулей, регистрацией публичных символов и разрешением внешних ссылок. Без учета этой особенности байткод-

ная реализация остается ограниченной и не отражает полный сценарий использования языка.

2.2. Существующая инфраструктура языка Lama

Существующая инфраструктура языка Lama организована вокруг компилятора, среды выполнения и стандартной библиотеки [8]. Точкой входа служит `Driver.ml`: он разбирает программу через `Language.run_parser` и в зависимости от параметров запуска выбирает один из путей исполнения или компиляции.

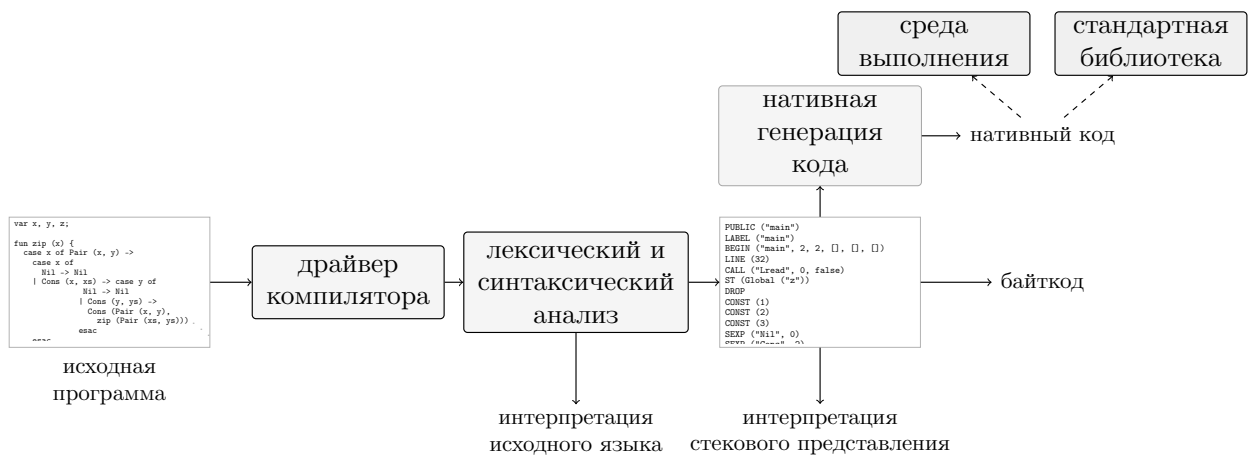


Рис. 1: Архитектура существующей инфраструктуры Lama на уровне основных представлений и путей исполнения.

По умолчанию и в режиме `-c` драйвер выбирает нативный бэкенд для архитектуры `X86_64`. Этот бэкенд также использует стековое представление: сначала программа преобразуется через `SM.compile`, а затем полученный стековый код транслируется в нативный код. В режиме `-b` то же стековое представление записывается как байткод через `SM.ByteCode.compile`. В режиме `-s` оно выполняется интерпретатором `SM.run`. Режим `-i` обходит стековую машину и запускает исходный уровень через `Language.eval`. Таким образом, `Language.ml` отвечает не только за лексический и синтаксический анализ программы, но и за отдельный путь интерпретации исходного языка, тогда как `SM.ml` является общей точкой для нативной генерации, байткодной генерации и стекового исполнения.

Нативная ветка связана с существующей средой выполнения. В репозитории есть отдельные исходные коды для 64-битной сборки, а реализация сборщика мусора находится в `gc.c`. Каталог `stdlib/` содержит модули стандартной библиотеки языка Лама; в существующей инфраструктуре они относятся прежде всего к нативной сборке программ, где поддерживаются импорты и связывание с объектными файлами стандартной библиотеки.

Байткод в этой инфраструктуре является сериализованной формой стекового представления. В нем уже нет многих поверхностных конструкций языка, но сохраняются операции загрузки констант, работы со стеком, обращения к локальным и глобальным значениям, вызова функций, создания замыканий, условных и безусловных переходов, а также сопоставления с образцом.

2.3. Ограничения существующего интерпретатора

Существующий путь интерпретации стекового представления реализован в `SM.ml` на `OCaml` [7, 9]. Он полезен как часть компиляторной инфраструктуры, однако не решает все задачи самостоятельного байткодного исполнения.

Первое ограничение связано с полнотой сценария запуска. В существующей инфраструктуре программы с импортами и стандартной библиотекой поддерживаются только нативной сборкой. Поэтому байткодное исполнение в общем случае требует отдельной поддержки загрузки модулей, регистрации публичных символов и разрешения межмодульных ссылок.

Второе ограничение связано с интеграцией со средой выполнения. Среда выполнения Лама задает представление значений, соглашения о выделении памяти, интерфейс сборщика мусора и набор встроенных функций. Интерпретатор на `OCaml` находится в другой среде исполнения, поэтому его прямое использование как байткодной виртуальной машины не дает той же модели взаимодействия со средой выполнения, что и нативный путь.

Наконец, существующий интерпретатор является частью компилятора, а не самостоятельной платформой для экспериментов с байтковым исполнением. Это ограничивает исследование таких аспектов, как внутреннее представление инструкций, диспетчеризация, организация стека и предварительная проверка байткода.

2.4. Подходы к организации виртуальных машин

В этом подразделе рассматриваются архитектурные решения, определяющие представление инструкций, модель вычислений, состояние исполнения и связь виртуальной машины со средой исполнения.

2.4.1. Представление исполняемых инструкций

При проектировании байткового исполнителя важен способ представления исполняемых инструкций [11]. Самый прямой вариант — выполнять байтовый поток непосредственно: цикл интерпретатора считывает очередной код операции, выбирает ветку обработки и декодирует операнды по мере исполнения. Такой подход прост и хорошо подходит для минимальной реализации, но часть работы повторяется при каждом выполнении инструкции. Кроме того, цели переходов, ссылки на функции и обращения к внешним символам приходится разрешать внутри основного цикла или рядом с ним.

Альтернативный подход состоит в предварительной конвертации байткода во внутреннее представление инструкций [12, 1]. На этапе загрузки байткод декодируется один раз, операнды приводятся к удобной для исполнения форме, цели переходов заменяются указателями или индексами внутренних инструкций, а внешние ссылки разрешаются через таблицу символов. После этого основной цикл работает уже с подготовленным массивом инструкций. Такой подход увеличивает объем предварительной обработки, но упрощает исполнение и позволяет раньше обнаруживать ошибки в байткоде.

2.4.2. Организация состояния исполнения

Другой важный выбор связан с моделью вычислений. Виртуальные машины часто разделяют на стековые и регистровые [11, 14]. В стековой машине операнды неявно берутся с вершины стека и результат также помещается на стек. Это делает инструкции компактными и хорошо соответствует байткоду языка Lаma, где многие операции уже имеют стековую форму. Регистровая машина хранит значения в явно адресуемых виртуальных регистрах; такой формат может уменьшать число операций со стеком, но требует другого представления кода и более сложного выбора операндов.

Модель вычислений связана с устройством состояния исполнения. Для языка с функциями, локальными переменными, аргументами, замыканиями и сборщиком мусора одного абстрактного стека операндов недостаточно. Виртуальная машина должна представлять кадры вызова, адрес возврата, границы локальных переменных, глобальные области модулей и значения, видимые сборщику мусора как корни. Возможны разные варианты: отдельный стек вызовов, единый стек значений с базовым указателем кадра или динамически выделяемые кадры. Для языка Lаma особенно важна совместимость со средой исполнения, поэтому состояние виртуальной машины должно быть организовано так, чтобы сборщик мусора мог корректно сканировать активные значения.

2.4.3. Взаимодействие со средой исполнения

Отдельную часть архитектуры образует взаимодействие с внешними функциями. Значительная часть встроенной функциональности уже реализована в среде исполнения языка Lаma, и виртуальная машина может переиспользовать эту логику. Возможны прямые вызовы заранее известных функций, таблица встроенных операций или более общий механизм разрешения внешних символов. Библиотека `libffi` показывает распространенный подход к такой задаче: она предоставляет переносимый интерфейс для вызова функций с соглашениями о вызовах, которые зависят от платформы и могут быть описаны во время вы-

полнения [13]. Для обратных вызовов в подобных интерфейсах также используются замыкания и trampлины, связывающие адрес, ожидаемый кодом на С, с обработчиком в среде исполнения. Для модульного байткода особенно важно, чтобы ссылки на внешние функции разрешались явно и ошибки отсутствующих символов обнаруживались до или во время подготовки программы к исполнению, а не приводили к неопределенному поведению.

2.5. Выводы

Проведенный обзор показывает, что для Lama недостаточно реализовать только набор обработчиков байткодных инструкций. На устройство байткодного исполнения влияют модульная система, существующая среда исполнения, представление значений языка и необходимость устойчивой обработки бинарного входа.

Ограничения существующего интерпретатора делают целесообразной реализацию новой виртуальной машины на С. Такая реализация может быть ближе к среде выполнения Lama, явно управлять стеком и кадрами вызова, использовать существующие встроенные функции и предоставлять основу для дальнейшего тестирования и оптимизации байткодного исполнения.

3. Архитектура

Виртуальная машина Lama реализована на С как отдельный исполнитель байткодных модулей. Ее архитектура описывает, как система получает байткод, загружает зависимости, связывает публичные символы, конвертирует байткод во внутреннее представление и исполняет программу с использованием существующей среды выполнения Lama. Входом системы является основной модуль `.bc` или имя модуля, а результатом работы является исполнение связанной программы языка Lama.

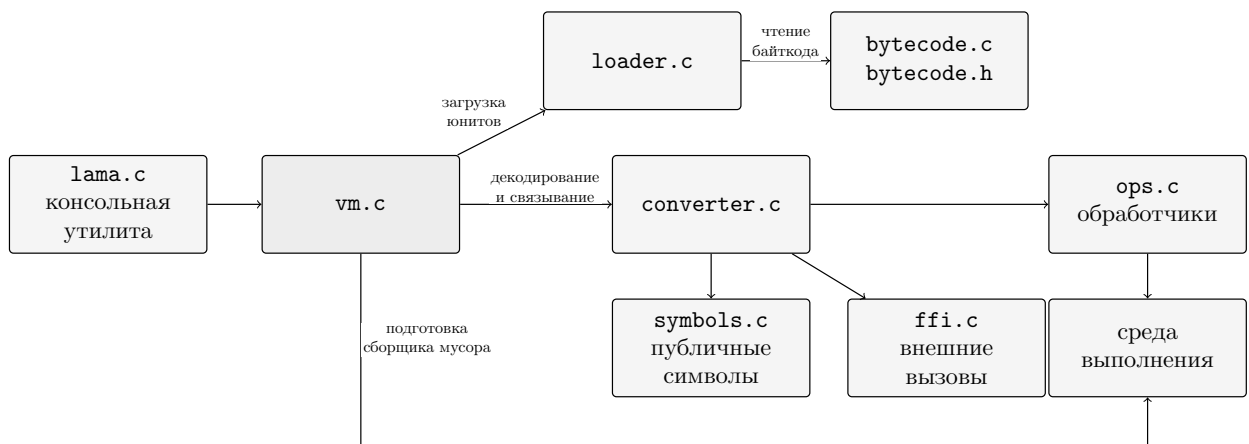


Рис. 2: Компонентная структура виртуальной машины Lama.

На рисунке 2 показаны основные компоненты виртуальной машины и их зоны ответственности. Внешний интерфейс сосредоточен в `lama.c`: этот файл разбирает аргументы командной строки, обрабатывает пути поиска байткодных модулей и управляет созданием и запуском виртуальной машины. Если пользователь передает путь к файлу `.bc`, из него выделяется имя основного модуля, а директория файла добавляется в список путей поиска.

Файл `vm.c` связывает остальные компоненты и хранит состояние виртуальной машины: загруженные байткодные модули, внутреннее представление байткода, точки входа, область глобальных значений, данные для внешних вызовов, стек и аргументы программы. Эти данные используются на границах между загрузкой, подготовкой программы и исполнением, поэтому `vm.c` выступает центральной точкой коор-

динации остальных модулей.

Загрузчик модулей в `loader.c` строит набор байткодных модулей, начиная с основного модуля. Он ищет файлы в заданных путях, рекурсивно обрабатывает импорты, отбрасывает повторные загрузки и обнаруживает циклические зависимости. Непосредственное чтение байткодного файла выполняется в `bytecode.c`: этот компонент разбирает секции файла, включая таблицу строк, список импортов, публичные символы, число глобальных значений и кодовую секцию.

Конвертер в `converter.c` отделяет бинарный формат `.bc` от исполняемого представления виртуальной машины. Он декодирует байткодные операции, проверяет переходы и согласованность стековой глубины, регистрирует публичные функции и глобальные значения через `symbols.c`, подготавливает внешние вызовы через `ffi.c` и выполняет релокации между модулями. Результат этого этапа — связанная программа в виде массива инструкций и их операндов, который уже не требует поиска целей переходов или межмодульных имен во время обычного исполнения.

Исполнение подготовленной программы выполняется обработчиками инструкций из `ops.c`. Эти обработчики работают с указателем текущей инструкции, стеком и базовым указателем кадра вызова. При операциях над объектами Lama, вызовах встроенных функций и работе сборщика мусора они обращаются к существующей среде выполнения Lama.

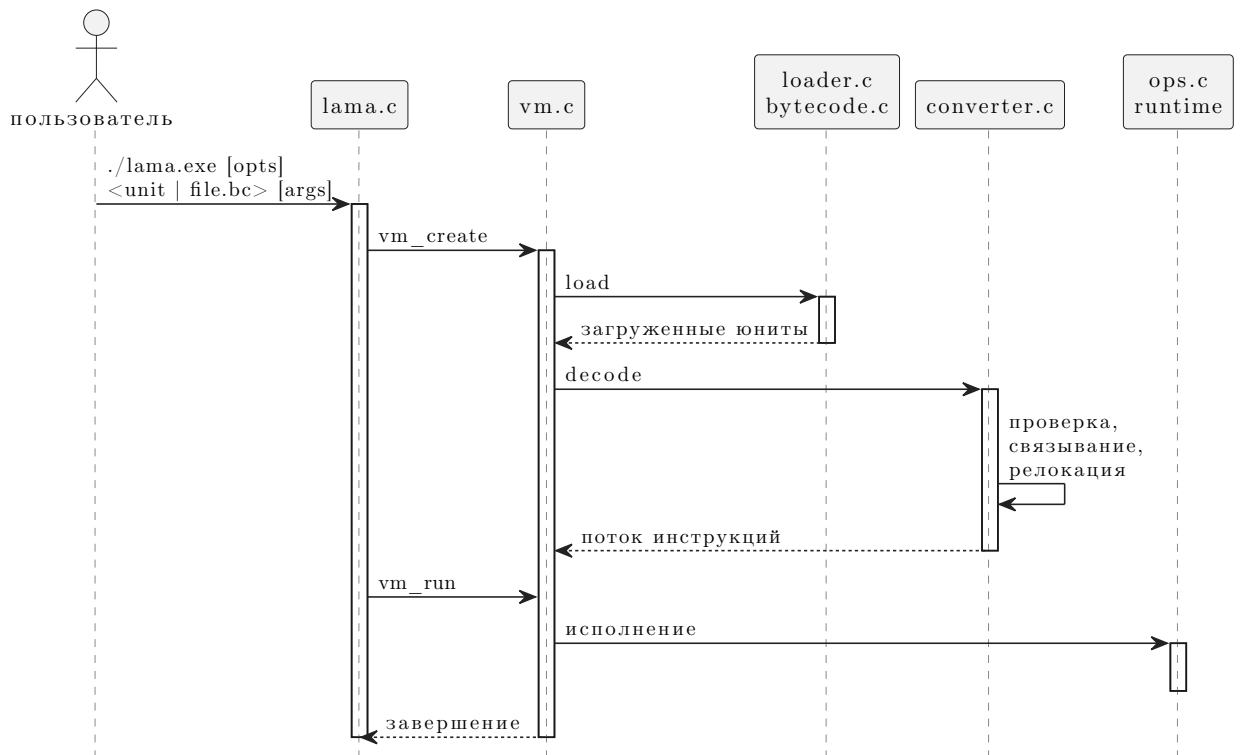


Рис. 3: Последовательность загрузки, декодирования и исполнения программы.

Рисунок 3 показывает тот же набор компонентов уже не как структуру, а как порядок работы при одном запуске программы. Вызов функции `vm_create` относится к фазе подготовки: в нее входят поиск байткодных файлов, чтение зависимостей, проверка, связывание и построение массива инструкций. После успешной подготовки управление возвращается в `lama.c`, который вызывает `vm_run`. На этой фазе `vm.c` инициализирует среду выполнения, передает аргументы программы и запускает исполнение с первой инструкции. После возврата из программы виртуальная машина завершает работу среды выполнения.

Ключевыми архитектурными решениями являются стековая модель исполнения, предварительная конвертация байткода в массив инструкций и их операндов и явное разделение загрузки, связывания и исполнения. Стековая модель соответствует байткоду, порождаемому компилятором Lama. Предварительная конвертация переносит проверку переходов, ссылок и стековых эффектов в фазу подготовки программы.

4. Описание реализации

В предыдущем разделе виртуальная машина рассматривалась как набор компонентов и поток управления между ними. В данном разделе описаны детали реализации, которые определяют поведение этих компонентов.

4.1. Прототип виртуальной машины

Разработка решения началась с прототипа исполнителя для существующего формата байткода Lama. Цель этого этапа состояла в том, чтобы проверить возможность непосредственного выполнения байткода, порождаемого существующим компилятором, и определить минимальный набор механизмов, необходимых для его интерпретации. Прототип поддерживал базовое стековое исполнение инструкций, передачу управления, вызовы функций и работу с основными значениями языка через существующую среду выполнения. Это позволило подтвердить практическую исполнимость байткодного пути и выделить ключевые требования к состоянию виртуальной машины.

Одновременно прототип показал ограничения исходного формата байткода. Для самостоятельного запуска многомодульных программ в нем отсутствовали сведения, необходимые для загрузки зависимостей, регистрации публичных символов и разрешения внешних ссылок. Кроме того, исполнение исходного байткода без предварительной подготовки оставляло проверки корректности и часть разрешения ссылок на этапе выполнения. Поэтому следующим шагом стало расширение формата байткода и фиксация его структуры в явной спецификации, после чего была построена итоговая реализация виртуальной машины, в которой фазы загрузки, связывания, подготовки программы и исполнения разделены явно.

4.2. Доработка генерации байткода

Виртуальная машина исполняет не исходное стековое представление компилятора, а байткодный файл. Поэтому часть работы была связана с тем, чтобы сделать этот файл пригодным для самостоятельной загрузки и связывания. В существующей инфраструктуре байткод фактически был сериализацией стековых инструкций одного модуля. Для виртуальной машины этого недостаточно: при запуске необходимо знать, какие модули импортируются, какие имена экспортируются наружу, сколько глобальных переменных требуется модулю и какие ссылки должны быть разрешены вне текущего файла.

Итоговая структура байткодного файла показана на рисунке 4. Все многобайтовые целые числа, как и раньше, записываются в порядке от младшего байта к старшему. Изменения касаются прежде всего заголовка, служебных секций и кодирования отдельных инструкций.

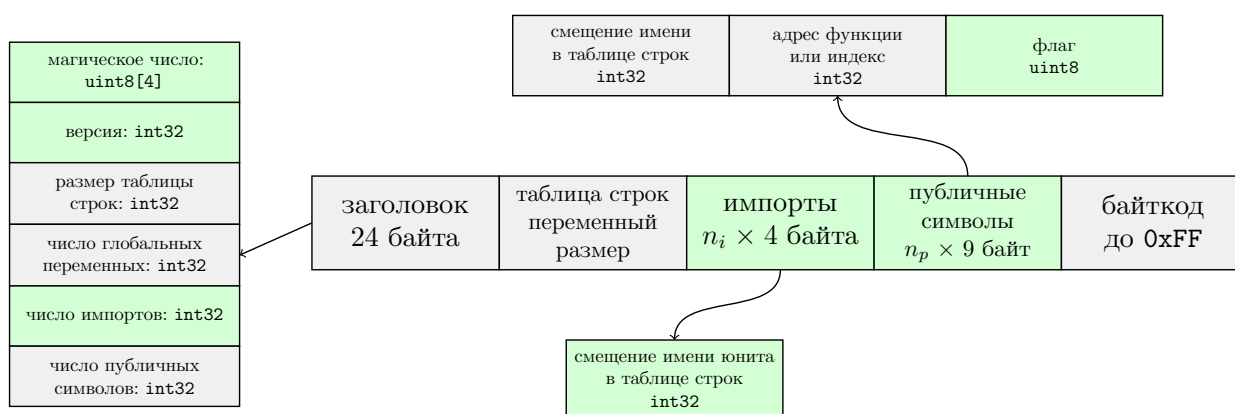


Рис. 4: Формат байткодного модуля Lama. Зеленым отмечены поля и секции, добавленные или измененные по сравнению с исходным форматом.

Первая группа изменений относится к общему устройству файла. Появление спецификации позволило явно зафиксировать свойства байткода, которые раньше были неявно заданы только компилятором. В заголовок были добавлены магическое число и версия формата байткода, чтобы загрузчик мог распознавать корректный вход и явно отличать версии байткода в будущем. Кроме того, в заголовке и теле файла появились данные, необходимые для модульной загрузки: число импор-

тов, число публичных символов и соответствующие секции `imports` и `publics`. Секция `imports` хранит имена импортируемых модулей через смещения в таблице строк. Секция `publics` содержит экспортируемые сущности модуля; по сравнению с исходным форматом в нее был добавлен флаг типа символа, различающий публичную функцию и глобальное значение. Это различие нужно при связывании, поскольку адрес исполняемого кода и индекс глобальной переменной обрабатываются по-разному.

Вторая группа изменений связана с самими инструкциями. Для ссылок на внешние сущности было введено единое кодирование отрицательными операндами через таблицу строк. Инструкции `CALL` и `CLOSURE` используют такие операнды для внешних функций, а `LD_GLO` и `ST_GLO` — для внешних глобальных значений. Неотрицательные значения по-прежнему обозначают локальные ссылки внутри текущего модуля.

Кроме того, были изменены отдельные инструкции исходного байт-кода. В `BEGIN_CLOSURE` был добавлен операнд, задающий число замыкаемых переменных и необходимый для валидации стека. Была добавлена инструкция `FAIL_KEEP` для сохранения семантики выполнения программы. Наконец, несколько операций исходного байткода, таких как `read`, `write`, `length` и `string`, в новой реализации отображаются на прямые вызовы функций существующей среды выполнения.

Еще одна доработка связана с именованием. Префиксы, используемые нативными бэкендами для ассемблерных символов функций, встроенных функций и глобальных значений, были оставлены на стороне генерации нативного кода. Стековое представление и байткод при этом работают с исходными именами сущностей. Благодаря этому байткод-ный формат не зависит от соглашений нативного генератора, а нативная сборка сохраняет свои прежние правила именования.

4.3. Загрузка и связывание модулей

Входом виртуальной машины является либо имя байткодного модуля, либо путь к файлу с расширением `.bc`. Во втором случае имя ос-

нового модуля получается из имени файла, а директория этого файла автоматически добавляется в пути поиска. Дополнительные директории задаются параметром `-I`.

Загрузка модулей строится вокруг импортов, записанных в байткоде. Для каждого модуля считывается список зависимостей, после чего загрузчик ищет соответствующие файлы в заданных директориях. При этом загрузчик поддерживает стек текущей рекурсии: если модуль пытается импортировать один из модулей, находящихся в процессе загрузки, такая ситуация распознается как циклическая зависимость и приводит к ошибке.

Результатом этой фазы является упорядоченный набор байткодных модулей. В него сначала попадают зависимости, а затем модуль, который от них зависит. Такой порядок упрощает дальнейшую подготовку программы: к моменту связывания основного модуля публичные символы импортированных модулей уже могут быть зарегистрированы.

Связывание исполняемых целей выполняется через релокации в `converter.c`. На этапе декодирования цель инструкции еще может быть известна только как смещение внутри текущего байткодного модуля, имя публичной функции другого модуля или имя внешней функции среды выполнения. Поэтому конвертер сначала записывает «заглушку» и добавляет запись релокации: индекс исправляемого слота, имя цели при необходимости и вид цели. В реализации различаются три вида таких записей: внутренняя цель текущего модуля, публичная функция другого модуля и «заглушка» вызова внешней функции.

Откладывание этого шага связано с устройством внутреннего формата программы. Каждый модуль сначала декодируется в собственный временный массив инструкций. Только после этого все такие массивы копируются в единый `all_code`, где адреса инструкций становятся стабильными. На финальном этапе релокации заменяют «заглушки» настоящими указателями: внутренние цели интерпретируются относительно начала своего модуля в `all_code`, межмодульные вызовы берут уже вычисленный индекс из таблицы символов, а внешние вызовы направляются на сгенерированную инструкцию интерфейса внешних функций.

Отдельно обрабатываются глобальные значения. Для инструкций загрузки и записи глобального значения конвертер обычно сразу вычисляет указатель на соответствующую область значений, поэтому общий механизм релокаций используется прежде всего для исполняемых целей: переходов, вызовов, замыканий и «заглушек» интерфейса внешних функций.

4.4. Конвертация байткода во внутреннее представление

После загрузки байткодного файла его кодовая секция преобразуется из последовательности байтов во внутренний массив инструкций и их операндов. Эта фаза отделяет внешний бинарный формат от представления, с которым работает исполнитель: код операции заменяется ссылкой на обработчик, операнды декодируются один раз и сохраняются в той же форме, что и инструкции.

Для каждой инструкции конвертер , считывает операнды и добавляет во временный массив один или несколько элементов внутреннего представления. Например, инструкция с числовым операндом преобразуется в элемент с обработчиком операции и следующий за ним элемент с уже разобранным числом в том же формате. Инструкции перехода и вызова требуют дополнительной обработки. Для внутренних переходов конвертер проверяет корректность целевого смещения в пределах текущего модуля. Если же окончательный адрес цели зависит от размещения кода в общей программе, конвертер сохраняет место, которое позднее уточняется на этапе релокации.

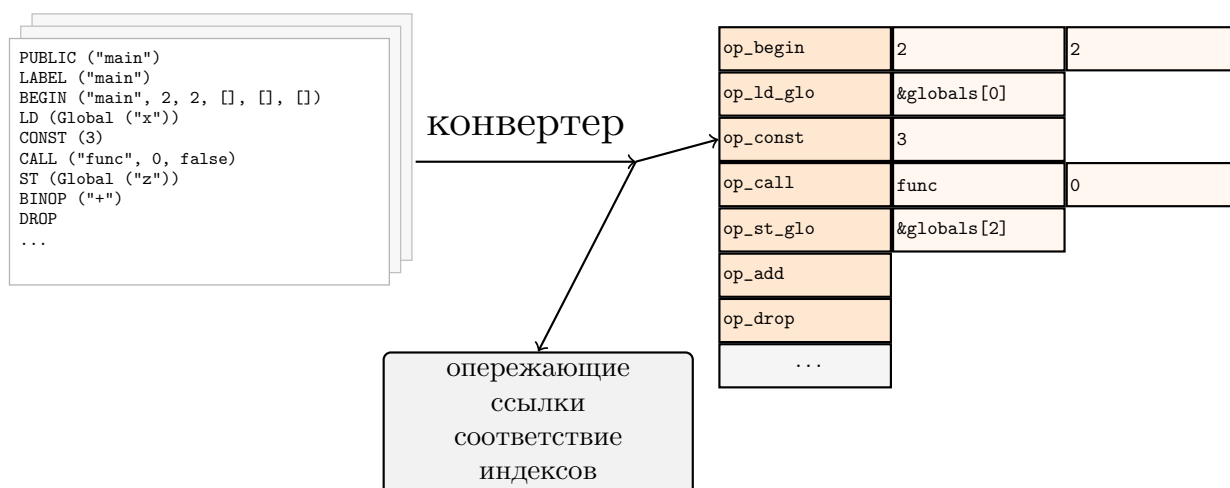


Рис. 5: Преобразование кодовой секции байткодного модуля во внутреннее представление инструкций.

На рисунке 5 показан процесс конвертации байткодных модулей. Конвертер последовательно обходит кодовую секцию каждого загруженного юнита, формирует временный массив инструкций и записывает релокации для тех операндов, которые нельзя заменить окончательным адресом сразу. Запись релокации содержит индекс исправляемого элемента внутреннего массива, имя цели при необходимости и вид ссылки: внутренняя цель текущего юнита, публичная функция другого юнита или внешняя функция среды выполнения.

На этом же этапе выполняются проверки, которые зависят от структуры кодовой секции. Конвертер проверяет, что цели переходов указывают на допустимые границы инструкций, а не в середину операнда. Для инструкций вызова и создания замыкания отдельно проверяется, что цель соответствует исполняемой позиции. Такие ошибки относятся к подготовке программы: их можно обнаружить до запуска основного цикла интерпретатора.

Результатом конвертации является временное внутреннее представление отдельного юнита. После обработки всех юнитов эти массивы копируются в общий массив инструкций виртуальной машины, где адреса становятся стабильными и могут быть использованы при окончательном разрешении переходов, вызовов и внешних ссылок.

4.4.1. Стек виртуальной машины

Стек виртуальной машины выделяется отдельно от системного стека. В текущей реализации `vm.c` создает область фиксированного размера с помощью `mmap`, использует верх этой области как базу стека, размещает глобальные переменные непосредственно перед ней и начинает исполнение с вершиной стека, указывающей на область глобальных значений. Эта же граница передается существующему сборщику мусора, чтобы он просматривал стек виртуальной машины.

Изначально рассматривался вариант с выделением стека через `alloca`. На практике он оказался ненадежным при взаимодействии со средой выполнения `Lama`: в области системного стека оставались устаревшие значения, которые сборщик мусора мог принять за корни. Из-за этого в множество достижимых значений попадали данные, уже не являвшиеся актуальными значениями программы, что приводило к падениям. Отдельная область, выделенная через `mmap`, делает границы сканирования явными и ограничивает их стеком виртуальной машины и глобальными переменными, предоставляя при этом динамическое расширение по мере исполнения программы.

4.4.2. Внутренний шитый код и внешние вызовы

Внутренний массив инструкций организован так, чтобы каждый элемент содержал либо ссылку на обработчик операции, либо ее уже подготовленный операнд. После выполнения очередной операции управление переходит к следующему элементу или к заранее вычисленной цели перехода. Такой подход уменьшает работу, выполняемую в интерпретации: исполнитель не декодирует байткод повторно, а работает с подготовленным представлением программы.

Схематично такой способ исполнения показан на рисунке 6. Подготовленное представление состоит из основного массива `all_code` и отдельного массива стартовых вызовов `entry_points`. Для каждого юнита в этом массиве хранится небольшой шаг запуска: вызов его основной функции, число аргументов, удаление результата и переход к следующему

щему юниту. Первый такой шаг указывает на начало основного кода первого юнита. Внутри `all_code` массив `insn` чередует элементы с указателями на обработчики и элементы с операндами; обработчик сам знает, сколько операндов необходимо прочитать.

Переход между обработчиками инструкций реализован как хвостовой вызов следующего обработчика. Для этого в макросах диспетчеризации используется атрибут компилятора `__attribute__((musttail))`: он требует, чтобы вызов следующего обработчика был скомпилирован как хвостовой переход. Без такого требования последовательное исполнение инструкций могло бы накапливать кадры на системном стеке и приводить к его переполнению на длинных программах. Кроме того, хвостовой переход уменьшает накладные расходы диспетчеризации, так как между обработчиками не выполняется обычная цепочка вложенных вызовов и возвратов.

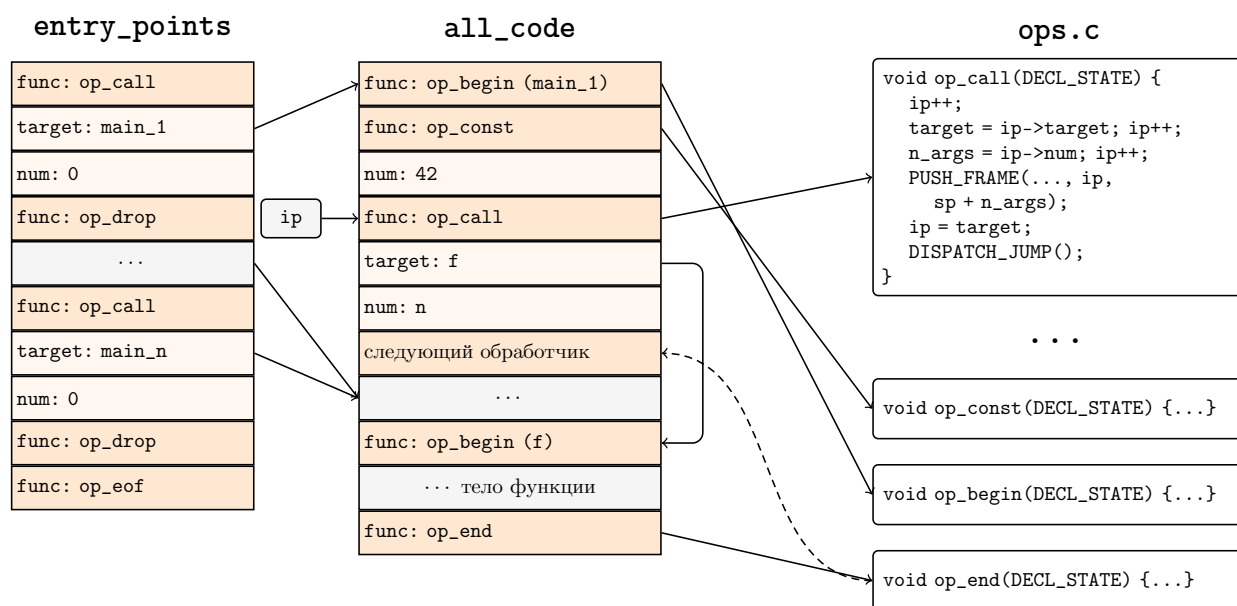


Рис. 6: Исполнение внутреннего шитого кода виртуальной машины.

Операции, создающие объекты Lama или обращающиеся к встроенным функциям, используют существующую среду выполнения. Перед такими вызовами виртуальная машина синхронизирует верхнюю границу своего стека со сборщиком мусора, чтобы значения, лежащие на стеке, учитывались как корни.

Внешние вызовы реализованы через отдельную инструкцию вызова, использующую интерфейс внешних функций. Имена внешних функций извлекаются из таблицы строк байткода и разрешаются через символы текущего процесса. Для обычных функций используется единый механизм вызова. Отдельно обрабатываются функции, которым среда выполнения передает массив аргументов, и вариативные функции наподобие форматированного вывода. Такое разделение позволяет сохранить совместимость с существующими соглашениями Lama, не добавляя специальные инструкции для каждой встроенной функции.

5. Тестирование и оценка производительности

В данном разделе собраны методы проверки разработанной виртуальной машины и оценка её производительности.

5.1. Методика тестирования

Тестирование виртуальной машины строилось как комбинация нескольких подходов, которые проверяют разные классы свойств. Регрессионные тесты фиксируют уже известные сценарии работы компилятора, стандартной библиотеки и среды выполнения. Такие тесты удобны для проверки повторяемости исправлений, но сами по себе покрывают только заранее выбранные случаи.

Для проверки корректности использовались два вида фаззинга. Первый подавал виртуальной машине повреждённый байткод и проверял, что ошибки в заголовках, операндах и ссылках не приводят к авариям, ошибкам памяти или неопределённому поведению. Второй порождал грамматически корректные программы Lama, компилировал их в байткод и в нативную программу и сравнивал результаты исполнения. Такая схема позволяла одновременно проверять устойчивость реализации к некорректным входам и согласованность её семантики с существующей реализацией языка. В таблице 1 приведены результаты данных методов тестирования.

Таблица 1: Сравнение результатов байткодного и грамматического фаззинга

Метрика	Байткодный фаззинг	Грамматический фаззинг
Длительность кампании, ч	17.6	20.6
Суммарная скорость, исп/с	1854.5	61.7
Общее число запусков	117 803 305	4 564 820
Покрытие строк, %	68.5	75.4
Исправления	8	1

Более подробно методика тестирования описана в предыдущей работе [15].

5.2. Оценка производительности

Целью данного раздела является оценка производительности реализованной виртуальной машины. В рамках эксперимента были поставлены следующие вопросы:

RQ1: Каковы временные накладные расходы виртуальной машины относительно нативного исполнения?

RQ2: Каковы накладные расходы виртуальной машины по памяти относительно нативного исполнения?

5.2.1. Набор данных и оборудование

Для проведения экспериментов были использованы пять программ на Lama разного объема и сложности:

- **Sort.lama** — сортировка пузырьком массива чисел
- **Ackermann.lama** — вычисление значений функции Аккермана
- **GenCyclicArray.lama** — построение и сравнение циклических массивов
- **Компилятор Lama на Lama** — в качестве входных данных будут использоваться его тесты:
 - **regression/** — основной набор регрессионных тестов компилятора Lama на корректность исполнения программ.
 - **regression/expressions** — автоматически сгенерированные программы, состоящие в основном из простых выражений.
 - **regression/deep-expressions** — автоматически сгенерированные программы с глубоко вложенными выражениями.

Эксперименты проводились на следующем тестовом оборудовании:

- ЦПУ: AMD Ryzen 7 6800H
 - каждый запуск привязывался к одному физическому ядру (без одновременной многопоточности)
- ОЗУ: DDR5 16 ГБ @ 6400 МГц
 - без подкачки
- ОС: Ubuntu 26.04 x86_64
- Компилятор gcc версии Ubuntu 15.2.0-16ubuntu1

Все измерения повторялись 20 раз. Для времени исполнения и пикового потребления памяти в качестве основной оценки использовалась медиана по запускам. Для времени дополнительно приводятся первый и третий квартили.

5.2.2. Эксперимент

5.3. Анализ выявленных ошибок и ограничений

TODO

Заключение

В ходе данной работы были решены следующие задачи.

1. Реализован прототип виртуальной машины для существующего формата представления байткода Lama
2. Доработаны формат представления байткода Lama и прототип виртуальной машины для возможности исполнения многомодульных программ
3. Реализована виртуальная машина на основе прототипа с предварительным анализом байткода для ускорения выполнения
4. Проверены корректность работы виртуальной машины и её производительность на существующих тестах

Ознакомиться с кодом можно по ссылке <https://github.com/ancavar/Lama/new/new-vm/>

Список литературы

- [1] Bell James R. Threaded code // *Commun. ACM*. — 1973. — June. — Vol. 16, no. 6. — P. 370–372. — URL: <https://doi.org/10.1145/362248.362270>.
- [2] Boulytchev Dmitry. — Lama Language Specification, 2025. — URL: <https://github.com/PLTools/Lama/blob/1.30/lama-spec.pdf>.
- [3] Compilers: Principles, Techniques, and Tools / Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman. — 2 edition. — Pearson, 2006. — ISBN: [9780321486813](https://www.isbn-international.org/product/9780321486813).
- [4] Cooper Keith D., Torczon Linda. Engineering a Compiler. — 3 edition. — Morgan Kaufmann, 2022.
- [5] Ierusalimschy Roberto, de Figueiredo Luiz Henrique, Celes Walde-
mar. — The Implementation of Lua 5.0, 2003. — URL: <https://www.lua.org/doc/jucs05.pdf> (дата обращения: 10 мая 2026 г.).
- [6] Lindholm Tim, Yellin Frank, Bracha Gilad et al. — The Java Virtual Machine Specification: Java SE 26 Edition, 2026. — February 17, 2026. URL: <https://docs.oracle.com/javase/specs/jvms/se26/html/> (дата обращения: 10 мая 2026 г.).
- [7] Lama Interpreter. — URL: <https://github.com/PLTools/Lama/blob/1.30/src/SM.ml> (дата обращения: 10 мая 2026 г.).
- [8] Lama Programming Language Repository. — URL: <https://github.com/PLTools/Lama> (дата обращения: 10 мая 2026 г.).
- [9] Leroy Xavier, Doligez Damien, Frisch Alain et al. — The OCaml System, Release 5.4: Documentation and User’s Manual, 2025. — October 9, 2025. URL: <https://ocaml.org/manual/> (дата обращения: 10 мая 2026 г.).

- [10] Python Software Foundation.— The Python Language Reference, 2026.— URL: <https://docs.python.org/3/reference/> (дата обращения: 10 мая 2026 г.).
- [11] Virtual machine showdown: Stack versus registers / Yunhe Shi, Kevin Casey, M. Anton Ertl, David Gregg // *ACM Trans. Archit. Code Optim.* — 2008.— January.— Vol. 4, no. 4.— 36 p.— URL: <https://doi.org/10.1145/1328195.1328197>.
- [12] dram. Threaded Code.— 2024.— URL: <https://dram.page/p/threaded-code/> (дата обращения: 2026-05-11).
- [13] libffi: A Portable Foreign Function Interface Library.— URL: <https://github.com/libffi/libffi> (дата обращения: 12 мая 2026 г.).
- [14] Šimek Bohuslav, Fiala Dalibor, Dostal Martin. Register-Based and Stack-Based Virtual Machines: Which Perform Better in JIT Compilation Scenarios? // *Software: Practice and Experience*. — 2025.— Vol. 55, no. 11.— P. 1896–1910.— <https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.70014>.
- [15] Парфенов Д. И. Тестирование и верификация виртуальной машины для языка Lama. — СПбГУ. — 2026.